Efficient and Tidy Manipulation of Annotated Matrix Data with plyxp

Justin T. Landis[1,2] Michael I. Love[2,3]

[1]Bioinformatics and Computation Biology Program, [2]Department of Genetics, [3]Department of Biostatistics

In order to manipulate high-dimensional omics data, such as bulk or single cell gene expression counts matrices, a bioinformatics analyst must typically learn domain-specific functions and syntax. These matrix-centric functions and syntax may not be as straightforward as working with tidy data analytic principles using tools such as dplyr applied to tabular data. We propose an expressive grammar for manipulating annotated matrix data, with syntax to compute on and update matrix data and tabular row and column metadata, including row-wise or column-wise grouped operations. This grammar works by defining multiple contexts, and providing pronouns for specific recall and assignment in these contexts. The plyxp package is implementation of this grammar for the R/Bioconductor ecosystem, with efficient abstractions for the SummarizedExperiment class. We demonstrate plyxp's efficiency compared to alternative approaches on data manipulation tasks requiring computation across contexts.

Introduction

High-throughput omics data are often summarized into one or more matrices representing counts or relative abundance of molecules in a set of experiments, as summarized across various genomic features (rows), and across different experiments or samples (columns). During subsequent analysis, it is critical to preserve links between rows and columns of the matrix data with relevant feature and sample metadata tables. For example, one may wish to divide a matrix of RNA-seq transcript counts by the transcript length (metadata associated with the transcript features along the rows) or by the library size (metadata associated with samples along the columns). This need for robust and efficient handling of annotated matrix data motivated the development of various data classes, including but not limited to the *ExpressionSet* [1] and the *SummarizedExperiment* [2] in R/Bioconductor, and *AnnData* [3] in Python. Such annotated matrix data classes and associated methods ensure that structural consistency is maintained between data and metadata during common operations such as subsetting, re-ordering, and row or column binding.

Operating on these data objects requires learning domain-specific functions and syntax. Performing complex operations such as grouping over sets of rows and columns and computing further data summarization may require writing additional control structures such as for loops or using less well-known functional programming helpers, which can obfuscate the purpose of analysis code. The *tidyomics* project was created within R/Bioconductor to help address this issue [4], providing methods and coding paradigms for working with common R data classes representing genomic ranges [5], [6], count matrices [7], and Seurat objects [8]. The project enables intuitive manipulation and visualization without requiring users to learn domain-specific syntax. *tidyomics* packages and workflows enable chained data-manipulative operations that resemble the style and framework of tidy data analysis [9] as implemented in *dplyr* [10] and other *tidyverse* [11]

packages, or chained operations for python *DataFrames* using *pandas* [12]. Due to the popularity of the *tidyverse* among newer R users, *tidyomics* increases the accessibility of omic data analysis.

Here, we propose a new expressive grammar for access and manipulation of annotated matrix data and a framework for efficiently performing the operations implied by the syntax. Data within the annotated matrix object are lazily bound to a series of environments that are evaluated when the user forces their symbols. Thus, a user has more freedom in how they choose to work with their data. We introduce the *plyxp* R package that implements this grammar, allowing the use of *dplyr* syntax with the Bioconductor *SummarizedExperiment* (SE) data class.

The *tidySummarizedExperiment* package, which was released with Bioconductor 3.12 in 2020, also provides *dplyr*-like access to SE objects within the *tidyomics* project. The *tidySummarizedExperiment* package allows users to approach the SE as an abstracted data frame, conceptually unwrapping the compact matrix representation into a tidy, long-form version. *tidySummarizedExperiment* facilitates data exploration and visualization, allowing datasets to be directly piped into *ggplot* plotting functions.

However some complex computations involving simultaneous manipulation of matrix values and annotation metadata were slower than necessary, a limitation addressable through data-masking strategies.

plyxp's use of data masking from the *rlang* [13] package provides gains in efficiency and is currently used to speed up existing functionality within *tidySummarizedExperiment*.

Further, *plyxp* reinforces the structure of the annotated matrix through additional syntax compared to *tidySummarizedExperiment*, often resulting in more verbose, but more specific evaluation.

Syntax

We describe a grammar in which users can access, modify, and append data across different contexts of a dataset, where here we mean "context" to refer to a collection of objects of the same length (for vectors) or dimension (for matrices), which collectively describe either features, samples, or a matrix of values over features and samples. For annotated matrix data with n rows and p columns, we define three contexts: the `assays` consist of one or more matrices of size $n \times p$; the `rows` consist of row metadata (a collection of vectors of length n describing the features); and the `cols` consist of column metadata (a collection of vectors of length p describing the samples/cells). For example, an object representing an RNA-seq experiment might have an `assays` context consisting of a counts matrix and a transcripts-per-million (TPM) matrix across n genes and p samples, a `rows` context with feature metadata such as alternative gene identifiers and gene length, and a `cols` context with sample metadata such as library size and sample condition group.

By default, expressions are evaluated against the named symbols within the `assays` context. The user can specify to evaluate against named symbols in the row or column metadata by decorating their expressions with sentinel functions, `rows(...)` or `cols(...)`. Beyond communicating where expressions are evaluated, this syntax also enforces where the resulting vector or matrix will be stored. For example, with the `mutate()` command, the new objects are bound to their respective context within

the object, but for the `filter()` command, instead of adding data, these results inform how the rows or columns should be subset.

```
data |>
  mutate(
    log10_counts = log10(counts + 1),
    cols(treated = (dex == "trt"))
  )
```

```
data |>
  filter(
    rows(gene_biotype == "lincRNA"),
    cols(avgLength >= 126)
  )
```

The above example demonstrates this grammar as implemented in *plyxp*.

The user computes the logarithm of the counts matrix plus a pseudocount, and creates a new logical variable in the column metadata, each of which are stored in their respective contexts. In the next step of the pipe, rows and columns are filtered based on available metadata about the genes and the samples. Here the base R `|>` symbol refers to piping the results of a function into the first place argument of the following function. As in *dplyr*, the `mutate()` verb implies an addition of an object (matrix or vector) to the corresponding evaluation context, whereas `filter()` subsets the data (described more later).

The *plyxp* syntax is designed to be convenient and flexible, providing users with multiple tools to access or manipulate their data across contexts. For example, consider the following:

```
data |>
  mutate(
    norm_cts = counts/.cols$size_factor,
    cts_bp = counts/.rows$gene_length
  )
```

Using pronouns `.cols` and `.rows`, the user accesses vectors from another context. A "pronoun" is a special object of the *rlang* package that provides an unambiguous symbol lookup into named lists or environments. In the above expression, an element-wise division of the counts matrix via data within the columns or rows is done by replicating the data to the appropriate size automatically.

Alternatively, in the computation of `gene_ave` in the following code chunk, the user employs the efficient row-wise means operator `rowMeans`. Alternatively, for `gene_ave2` the expression `.assays$counts` within the row context is resolved to a list where each element is a row of the counts matrix. A list output for the `.assays` pronoun offers flexibility as lists can be supplied to the apply family of functions or using *purrr* [14] as shown below.

```
data |>
  mutate(
    rows(
      gene_ave = rowMeans(.assays_asis$counts),
      gene_ave2 = map_dbl(.assays$counts, mean)
    )
  )
```

In the example code above, the `data` object can be instantiated from any *SummarizedExperiment* using the `new_plyxp()` function (see Supplementary Note for more details).

Semantics

plyxp is an implementation of this grammar for annotated matrix data stored in R/Bioconductor's *SummarizedExperiment* class, allowing the use of *dplyr* verbs with the following expected outcomes and behaviors. *plyxp* enforces conventions that eliminate ambiguity while providing flexibility in terms of data shape and computation:

1. Expressions are scoped explicitly to an evaluation context.
2. Evaluation contexts are connected via pronouns, which provide both an `asis` and size-stable version of the requested data.

3. *dplyr* verbs (`mutate()`, `filter()`, etc.) only support expressions that would result in the same structure of object as input (SE object).

Evaluation contexts and their connection via pronouns are diagrammed in Figure Figure 1. We note that *plyxp* relies on Bioconductor-native calls, e.g. subsetting with square brackets `[,]`, and is thus never faster than using the SE interface directly. There will always be some overhead in generating the necessary abstraction to enable implicit references into the object as required by *dplyr*. However, we aim that *plyxp* always be as close to Bioconductor-native speed as possible, via line profiling of the function definitions using *profvis*.

Pronouns and Data Masks

plyxp enables expressive evaluation by leveraging *rlang* data masks, a series of basic R environments that are evaluated with the *rlang* tidy evaluation engine [13, H. Wickham [15]]. *plyxp* exploits several utilities from the *rlang* package, such as lazy evaluation and *rlang* pronouns, to manage these environments. Here, "lazy evaluation" refers to expressions whose computation is delayed until the value is accessed. A developer may choose any number of environments for their data mask that are necessary for their use case. With multi-context scenarios that require multiple data masks, as with annotated matrix data, we suggest the final parent ancestor environment of the mask is reserved for binding *rlang* pronouns. This suggestion is motivated by R's lexical scoping, which allows variable bindings to be searched through environment parents during evaluation [16]. Placing pronouns in the final parent of the mask, *i.g.* the final environment prior to the calling environment, will ensure the user's data within the context are prioritized over potential access to other contexts. The *rlang* pronouns can be conceptually thought of as a pointer to another environment. *rlang* pronouns are utilized within the *dplyr* package to disambiguate where to find a symbol, in either the calling environment or user's data with `.env`and `.data` pronouns respectively.

In *plyxp*, these pronouns are used to connect each context through two pronouns, an `asis` variant and a "size-stable" variant. The `asis`-variant is to provide the user with access to the actual data stored on the object, as there may be efficient ways to operate on its native shape *e.g.* the row-wise and column-wise matrix operations in base R and *matrixStats*. The size-stable variant provides the requested data reshaped to match the size of the current context. The size-stable variant is intended to reduce friction from pulling data from other contexts, as there is no guarantee the target data`asis` will match the expected output size of the evaluation context.

With annotated matrix data, there are predictable transformations that may be used to guarantee size stability. For example, consider requesting row or column data from within the `assays` evaluation context. To make the row data size stable, we need only to replicate the target vector as many times as the number of columns in our matrix context, and then concatenate these vectors. For column data within the `assays` evaluation context, we replicate each element of the target vector as many times as the number of rows in our matrix context. Requesting assay data from within the `rows` or `cols` evaluation context, we may reshape the matrix by returning a list where each element is a row or column of the matrix, respectively.

Grouping

In *dplyr*, specifying groups with `group_by()` augments the behavior of further *dplyr* verbs. *dplyr* grouping operations traditionally in-

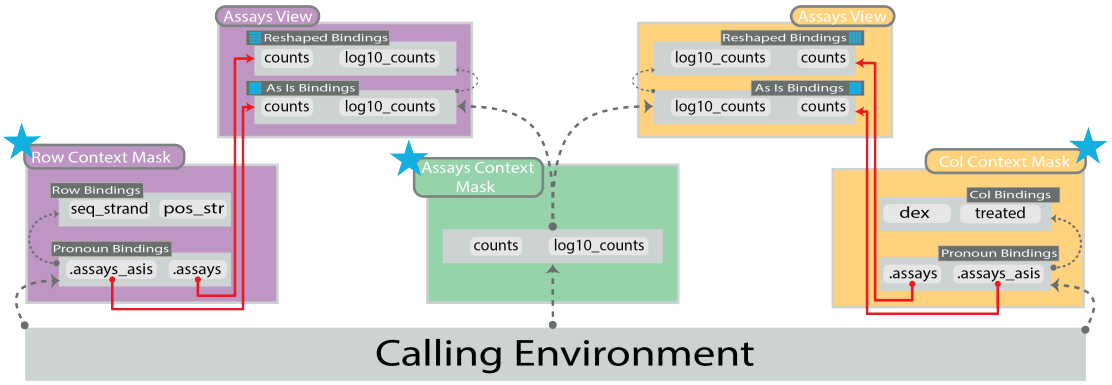


Figure 1: Diagram of the data masking structure in *plyxp*. Grey rectangles represent environments. Parent environments are linked to child environments by dotted lines where the circle indicates the parent and the arrow indicates the child environment. Full environment hierarchies are not explicitly shown. Example variable bindings are shown within environments. Data mask contexts are grouped by color where the `assays`, `rows`, and `cols` contexts are represented by the green, purple, and yellow blocks respectively. Red lines show connections from pronouns of one context to environments that are children of other contexts. For brevity, only extended environments from the `assays` context into the `rows` and `cols` contexts are shown. Entry points for each context is indicated by the blue star. Through *rlang*'s tidy evaluation engine, symbols within the data mask are evaluated before searching the calling environment.

clude additional metadata on the target object describing how the data should be partitioned. This strategy works well for data structures that are rectangular, but may lead to ambiguities for data structures that deviate from this structure, such as annotated matrix data. For tabular data, there is only one dimension (along the rows) in which those data can be partitioned, and no matter how the rows are partitioned, a new table can be constructed from the results. This is not true for matrices. One such example would be to attempt to group by matrix values > 100. Here, there is no guarantee the partitioning of arbitrary elements of a matrix would create valid submatrices. It is for this reason that *plyxp* follows strict heuristics for grouping and only allows groupings that partition rows or columns of the matrix.

This restriction coincides with the natural extension of applying grouping to tabular data from *dplyr*. We may conceptually regard the row data and the column data as independent and grouped as normal tabular structures. The row data and the column data are related to one another through the matrix, and thus the matrix observes all grouping partitions imposed on both dimensions. Since the row data and the column data groupings are independent, the `rows` and `cols` contexts only consider their own respective grouping during evaluation. In other words, the `assays` context will always respect groupings about the `rows` and `cols` contexts, but the `rows` context drops any `cols` grouping and `cols` context drops any `rows` grouping.

Performance Comparison

All evaluation within *plyxp* is intended to take place within one of the specified contexts mentioned previously. Due to the particular shape of annotated matrix data, each context has a

unique notion of size and a unique set of symbols within. Additional syntax is used to declare where an expression evaluates and there is no additional cost to accessing data `asis`. An alternative to this approach would be to force all variables within the same context, such as an instantiated or abstracted *data.frame* where all contexts are joined together.

A single context would require variables within the assays to be unwound into one dimensional vectors, and data within the row and column metadata tables to be replicated appropriately. Additional performance considerations are discussed in more detail in the supplemental note. When comparing between the two approaches, namely *plyxp* compared to annotated matrix data unwound into a *data.frame*, *plyxp* runs ~ 14 times faster when scaling the counts matrix by gene length using the *airway* dataset [17].

Conclusion

plyxp provides a framework for operating on annotated matrix data. As data masking was possible via the *rlang* package, we were able to introduce novel syntax for operating on SE objects within Bioconductor that is ergonomically inspired by *dplyr* style and unambiguously expressive. *tidySummarizedExperiment* has incorporated *plyxp* methods internally for some operations as of Bioconductor 3.22 release in Fall 2025, such that users of both *plyxp* and *tidySummarizedExperiment* benefit from the performance gains described above, as compared to operations that would require creation of a long form tabular representation of the annotated matrix data. We imagine that the grammar described here could be applied to manipulate *AnnData* with *pandas* methods. We plan to release the internals of *plyxp* that construct and manage the data masks so that developers can choose to create their own *dplyr* style for additional R classes.

Code Availability

The *plyxp* software package is available from Bioconductor, installable with `BiocManager::install()`, and at https://github.com/jtlandis/plyxp. All code used in this paper, including comparisons of run time to alternative implementations and reproducible code chunks appearing above, is available at https://github.com/jtlandis/plyxp_paper

Bibliography

[1] R. C. Gentleman *et al.*, "Bioconductor: open software development for computational biology and bioinformatics," *Genome Biology*, vol. 5, no. 10, p. R80, Sep. 2004.

[2] W. Huber *et al.*, "Orchestrating high-throughput genomic analysis with Bioconductor," *Nature Methods*, vol. 12, no. 2, pp. 115–121, Feb. 2015.

[3] I. Virshup, S. Rybakov, F. J. Theis, P. Angerer, and F. A. Wolf, "anndata: Access and store annotated data matrices," *Journal of Open Source Software*, vol. 9, no. 101, p. 4371, 2024, doi: 10.21105/joss.04371.

[4] W. J. Hutchison *et al.*, "The tidyomics ecosystem: enhancing omic data analyses," *Nature Methods*, vol. 21, no. 7, pp. 1166–1170, Jul. 2024.

[5] S. Lee, D. Cook, and M. Lawrence, "plyranges: a grammar of genomic data transformation," *Genome Biology*, vol. 20, no. 1, p. 4, Jan. 2019, doi: 10.1186/s13059-018-1597-8.

[6] S. Lee, M. Lawrence, and M. I. Love, “Fluent genomics with plyranges and tximeta,” *F1000Res.*, vol. 9, p. 109, Feb. 2020.

[7] S. Mangiola, R. Molania, R. Dong, M. A. Doyle, and A. T. Papenfuss, “tidybulk: an R tidy framework for modular transcriptomic data analysis,” *Genome Biol.*, vol. 22, no. 1, p. 42, Jan. 2021.

[8] S. Mangiola, M. A. Doyle, and A. T. Papenfuss, “Interfacing Seurat with the R tidy universe,” *Bioinformatics*, vol. 37, no. 22, pp. 4100–4107, Nov. 2021.

[9] H. Wickham, “Tidy Data,” *Journal of Statistical Software*, vol. 59, no. 10, pp. 1–23, 2014, doi: 10.18637/jss.v059.i10.

[10] H. Wickham, R. François, L. Henry, K. Müller, and D. Vaughan, “dplyr: A Grammar of Data Manipulation.” 2023. doi: 10.32614/CRAN.package.dplyr.

[11] H. Wickham *et al.*, “Welcome to the tidyverse,” *Journal of Open Source Software*, vol. 4, no. 43, p. 1686, 2019, doi: 10.21105/joss.01686.

[12] W. McKinney, *Python for data analysis 3e*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.

[13] L. Henry and H. Wickham, “rlang: Functions for Base Types and Core R and 'Tidyverse' Features.” 2026. doi: 10.32614/CRAN.package.rlang.

[14] H. Wickham and L. Henry, “purrr: Functional Programming Tools.” 2025. doi: 10.32614/CRAN.package.purrr.

[15] H. Wickham, *Advanced R, second edition*, 2nd ed. in Chapman & Hall/CRC The R Series. Philadelphia, PA: Chapman & Hall/CRC, 2019.

[16] R Core Team, “R: A Language and Environment for Statistical Computing.” Vienna, Austria, 2025. [Online]. Available: https://www.r-project.org/

[17] B. E. Himes *et al.*, “RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells,” *PLOS One*, vol. 9, no. 6, p. e99625, Jun. 2014.